

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..30.1

SETUP

 Copy as Markdown

Set up the OAuth2 utility

The Serverpod Auth module provides generic OAuth2 utilities that simplify implementing custom identity providers. These utilities handle the complex OAuth2 authorization code flow with PKCE (Proof Key for Code Exchange), allowing you to integrate any OAuth2-compliant provider without dealing with low-level protocol details.

The OAuth2 utility consists of client-side and server-side components that work together to securely authenticate users:

- **Client-side** (`OAuth2PkceUtil`): Manages the authorization flow in your Flutter app, handling browser redirects and PKCE challenge generation.
- **Server-side** (`OAuth2PkceUtil`): Exchanges authorization codes for access tokens on your backend.

INFO

The [GitHub provider](#) is built using these utilities, serving as a reference implementation for developers creating custom providers.

Understanding OAuth2 with PKCE

OAuth2 with PKCE is an authorization protocol that allows users to grant your application access to their data without sharing passwords. The PKCE extension adds an additional security layer, particularly important for mobile and public clients.

The OAuth2 flow

Here's how the complete flow works:

1. **Generate Code Verifier:** Client generates a random cryptographic string (code verifier).
2. **Generate Code Challenge:** Client creates a SHA-256 hash of the verifier (code challenge).



3. **Authorization Request:** Client redirects user to provider with the code challenge.
4. **User Authorizes:** User logs in and grants permissions.
5. **Receive Code:** Provider redirects back with an authorization code.
6. **Token Exchange:** Client sends code + verifier to your backend.
7. **Backend Exchange:** Backend exchanges code + verifier for access token.
8. **Access Protected Resources:** Use access token to fetch user information.

PKCE ensures that even if an attacker intercepts the authorization code, they cannot exchange it for an access token without the original code verifier.

Server-side implementation

Configuration

The OAuth2 utility requires a configuration object that defines how your server communicates with the OAuth2 provider's token endpoint. Create a server-side configuration for token exchange:

```
import 'package:serverpod_auth_idp_server/core.dart';

final config = OAuth2PkceServerConfig(
  // Token endpoint URL for exchanging authorization codes
  tokenEndpointUrl: Uri.https('oauth.provider.com', '/oauth/token'),

  // OAuth client ID (must match client-side)
  clientId: pod.getPassword('myProviderClientId')!,

  // OAuth client secret (keep secure!)
  clientSecret: pod.getPassword('myProviderClientSecret')!,

  // Function to parse token response from provider
  parseTokenResponse: (data) {
    // Your parse logic here
  },

  // Optional: Where to send credentials (default: header)
  credentialsLocation: OAuth2CredentialsLocation.header,

  // Optional: Custom parameter names for credentials
  clientIdKey: 'client_id',
  clientSecretKey: 'client_secret',

  // Optional: Custom headers for token requests
  tokenRequestHeaders: {
    'Accept': 'application/json',
    'Content-Type': 'application/x-www-form-urlencoded',
  },

  // Optional: Additional parameters for token exchange
  tokenRequestParams: {
    'grant_type': 'authorization_code',
```

```
},  
);
```

INFO

The `credentialsLocation` parameter controls how your client credentials are sent to the OAuth2 provider:

- **Header mode (recommended):** Credentials are placed in the `Authorization` header using HTTP Basic authentication. This follows RFC 6749 and is generally more secure, since sensitive values don't appear in the request body or logs.
- **Body mode:** Credentials are sent as form parameters in the request body. Use this only if your provider doesn't support header-based authentication.

Exchanging tokens

Using the previously created `config` object, create the `OAuth2PkceUtil` on your endpoint to exchange the authorization code:

```
import 'package:serverpod/serverpod.dart';  
import 'package:serverpod_auth_idp_server/core.dart';  
  
import '../generated/protocol.dart';  
  
class MyProviderIdpEndpoint extends IdpBaseEndpoint {  
  final oauth2Util = OAuth2PkceUtil(config: config);  
  
  /// Required by IdpBaseEndpoint: report whether the signed-in user already  
/// has an account with this provider. Query the account model your provider  
/// defines (see the full walkthrough for the model definition).  
  @override  
  Future<bool> hasAccount(Session session) async {  
    final authUserId = session.authenticated?.authUserId;  
    if (authUserId == null) return false;  
    return await MyProviderAccount.db.findFirstRow(  
      session,  
      where: (t) => t.authUserId.equals(authUserId),  
    ) !=  
      null;  
  }  
  
  Future<AuthSuccess> authenticate(  
    Session session, {  
    required String code,  
    required String codeVerifier,  
    required String redirectUri,  
  }) async {  
    try {  
      // Exchange authorization code for token response  
      final tokenResponse = await oauth2Util.exchangeCodeForToken(  

```

```

        code: code,
        codeVerifier: codeVerifier,
        redirectUri: redirectUri,
    );

    // Fetch user information from provider using access token
    final userInfo = await _fetchUserInfo(session, tokenResponse.accessToken);

    // Authenticate (find existing account or create new user)
    final account = await _authenticate(session, userInfo);

    // Issue authentication token to client
    return await _issueToken(
        session,
        authUserId: account.authUserId,
        scopes: account.scopes,
    );
} on OAuth2InvalidResponseException catch (e) {
    session.log('Invalid token response: ${e.message}');
    throw Exception('Authentication failed');
} on OAuth2MissingAccessTokenException catch (e) {
    session.log('Missing access token: ${e.message}');
    throw Exception('Authentication failed');
} on OAuth2NetworkErrorException catch (e) {
    session.log('Network error: ${e.message}');
    throw Exception('Network error during authentication');
}
}

Future<Map<String, dynamic>> _fetchUserInfo(
    Session session,
    String accessToken,
) async {
    // Fetch user data from provider's API using the access token
}

Future<_AccountResult> _authenticate(
    Session session,
    Map<String, dynamic> userInfo,
) async {
    // Find the existing provider account or create a new auth user from the
    // provider's user info, and return its authUserId with the scopes to grant.
}

Future<AuthSuccess> _issueToken(
    Session session, {
    required UuidValue authUserId,
    required Set<Scope> scopes,
}) async {
    // Issue Serverpod authentication token for the authenticated user
}

/// What _authenticate resolves: the linked auth user and the scopes to grant.
typedef _AccountResult = ({UuidValue authUserId, Set<Scope> scopes});

```

Exception handling

The server-side utility throws these exceptions:

Exception	Description	Typical Cause
<code>OAuth2InvalidResponseException</code>	Invalid response from provider	HTTP errors, malformed JSON
<code>OAuth2MissingAccessTokenException</code>	Access token not in response	Provider didn't return token
<code>OAuth2NetworkErrorException</code>	Network failure	Timeout, connection issues
<code>OAuth2UnknownException</code>	Unexpected error	Unknown problems

Client-side implementation

Configuration

The client-side OAuth2 utility also requires a configuration object that defines your provider's OAuth2 endpoints and authorization parameters. Create a client-side configuration:

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

final config = OAuth2PkceProviderClientConfig(
  // Your provider's authorization endpoint
  authorizationEndpoint: Uri.https('oauth.provider.com', '/oauth/authorize'),

  // OAuth client ID from your provider
  clientId: 'your-client-id',

  // Callback URI registered with your provider
  redirectUri: 'myapp://auth-callback',

  // URL scheme for the callback
  callbackUrlScheme: 'myapp',

  // Default permission scopes to request
  defaultScopes: ['profile', 'email'],

  // Additional query parameters for authorization request
  additionalAuthParams: {
    'response_mode': 'query',
  },

  // Separator for joining scopes (default: ' ')
  scopeSeparator: ' ',
```

```
// Enable state parameter for CSRF protection (default: true)
enableState: true,

// Enable PKCE for OAuth2 flow (default: true)
enablePKCE: true,
);
```

Initiating authorization

Using the previously created `config` object, create an `OAuth2PkceUtil` instance to start the authorization flow:

```
final oauth2Util = OAuth2PkceUtil(config: config);

try {
  final result = await oauth2Util.authorize(
    // Optional: override default scopes
    scopes: ['profile', 'email'],
  );

  // The authorization code to exchange for an access token
  final code = result.code;

  // The PKCE code verifier (required for token exchange)
  final codeVerifier = result.codeVerifier;

  // Send both to your backend. codeVerifier is null when the provider does
  // not use PKCE, so guard it before calling an endpoint that requires it.
  if (codeVerifier == null) {
    throw StateError('The provider did not return a PKCE code verifier.');
```

```
}
```

Exception handling

The client-side utility throws specific exceptions to help you handle different error scenarios:

Exception	Description	Typical Cause
<code>OAuth2PkceStateMismatchException</code>	State validation failed	Possible CSRF attack or browser issue
<code>OAuth2PkceMissingAuthorizationCodeException</code>	No authorization code received	Provider didn't return expected code
<code>OAuth2PkceProviderErrorException</code>	Provider returned error response	Invalid credentials, rate limiting
<code>OAuth2PkceUnknownException</code>	Unexpected error occurred	Network issues, unknown problems, and a cancelled sign-in

Platform-specific configuration

The OAuth2 utility uses the `flutter_web_auth_2` package under the hood, which requires platform-specific setup.

iOS and macOS

There is no special configuration needed for iOS and macOS for "normal" authentication flows. However, if you are using **Universal Links** on iOS, they require redirect URIs to use **https**. Follow the instructions in the `flutter_web_auth_2` documentation.

Android

Add the callback activity to your `AndroidManifest.xml`:

```
<manifest>
  <application>

    <activity
      android:name="com.linusu.flutter_web_auth_2.CallbackActivity"
      android:exported="true">
      <intent-filter android:label="flutter_web_auth_2">
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

```
<category android:name="android.intent.category.BROWSABLE" />
<!-- Replace with your actual callback URL scheme -->
<data android:scheme="myapp" />
</intent-filter>
</activity>

</application>
</manifest>
```

Web

Web sign-in needs the shared callback page that hands the OAuth2 result back to your app. Set it up once as described in [Web callback page](#) (`auth.html`). The same page serves every provider built on the OAuth2 utility, as long as your redirect URIs point to it, for example `https://yourdomain.com/auth.html`.

Complete example of a custom provider

For a full end-to-end implementation of a custom OAuth2 provider (server configuration, client setup, and integration of all components), see the [Complete Example](#) page.

Best practices

Security considerations

1. **Always Use PKCE:** Keep `enablePKCE: true` in your client configuration. PKCE protects against authorization code interception attacks.
2. **Validate State Parameter:** Keep `enableState: true` to prevent CSRF attacks. The state parameter ensures the authorization response matches your request.
3. **Secure Client Secret:** Never expose your client secret in client-side code. Store it securely in `passwords.yaml` or environment variables on the server.
4. **Use HTTPS:** Always use HTTPS URLs for production endpoints. Only use HTTP for local development.
5. **Validate Redirect URIs:** Ensure redirect URIs in your code exactly match those registered with your OAuth2 provider.

Error handling

1. **Catch Specific Exceptions:** Handle each exception type appropriately rather than using generic catch-all handlers.
2. **Log Securely:** Log errors for debugging but never log sensitive data like tokens or secrets.
3. **User-Friendly Messages:** Show clear, actionable error messages to users without exposing technical details.